202401471_Lab05 ICS

Dhirubhai Ambani University Technology

Formerly DA-IICT

Introduction to Communication System[CT216].

I,Khushi Shah,Student Id: 202401471,declare that:

- The work that I am presenting is my own work and I have not copied the work that someone else has done.
- The concepts, understanding, and insights I will be describing are my own understood during ongoing classes.
- Wherever I have relied on an existing work from a source, I have provided a proper reference citation.
- I make this pledge truthfully. I know that violation of this solemn pledge can carry consequences.

1 Programming and Simulation

1.1 Bernoulli Random Variable

```
clc; clear; close all;
p = 0.5;
% theoretical mean and variance: E[X]=p, Var[X]=p*(1-p)
E_theory   = p;
Var_theory = p*(1-p);
fprintf('Theoretical: E[X]=%.4f, Var[X]=%.4f\n', E_theory, Va
r_theory);
% simulate for Nsim = 10, 100, 1000
Nsim_vals = [10, 100, 1000];
for n = 1:length(Nsim_vals)
Nsim = Nsim_vals(n);
X    = double(rand(1,Nsim) < p);
fprintf('Nsim=%4d: sim E[X]=%.4f, sim Var[X]=%.4f\n', Nsim, m
ean(X), var(X));
end
```

```
Output :
Theoretical: E[X]=0.5000, Var[X]=0.2500
Nsim=  10: sim E[X]=0.3000, sim Var[X]=0.2333
Nsim= 100: sim E[X]=0.4400, sim Var[X]=0.2489
Nsim=1000: sim E[X]=0.4970, sim Var[X]=0.2502
```

1.2 Product Encoder, BEC, and Decoder

```
clc; clear; close all;
K = 4; N = 9;
k = sqrt(K); n = sqrt(N);
q = 0.3;
```

```
% Q1: Generate K message bits m ~ Bern(0.5)
m = double(rand(1,K) < 0.5);
fprintf('Message bits m = '); disp(m);

% Q2: Encoder - build N=9 encoded bits array c
% Arrange K=4 info bits in k x k (2x2) grid, add parity row a
nd column
M = reshape(m, k, k);                                          %
2x2 info bit matrix
% parity: XOR along rows and columns (even parity)
row_par = mod(sum(M, 2), 2);                                   %
2x1 row parities
M_ext   = [M, row_par];                                        %
2x3 extended with row parity
col_par = mod(sum(M_ext, 1), 2);                               %
1x3 col parities
C       = [M_ext; col_par];                                    %
3x3 codeword matrix
c       = C(:)';                                               %
N=9 encoded bits, row-major
fprintf('Codeword c = '); disp(c);

% Q3: BEC channel - generate N erasure flags y ~ Bern(q)
y = double(rand(1,N) < q);                                     %
1=erased, 0=received
fprintf('Erasure flags y = '); disp(y);

% Q4: Apply erasures to get received word r (-1 denotes erase
d bit)
r = c;
r(y == 1) = -1;                                                %
-1 marks erased positions
fprintf('Received r = '); disp(r);

% Q5: Iterative row-column decoder
R = reshape(r, n, n);                                          %
```

```
reshape to 3x3 matrix

% iterate up to n times (enough for any recoverable pattern)
for iter = 1:n
    % row-by-row: if exactly one -1 in a row, recover it by X
OR of known bits
    for i = 1:n
        row = R(i,:);
        if sum(row == -1) == 1                              %
exactly one erasure in row
            R(i, row == -1) = mod(sum(row(row ~= -1)), 2); %
recover by even parity
        end
    end
    % column-by-column: same logic
    for j = 1:n
        col = R(:,j);
        if sum(col == -1) == 1                              %
exactly one erasure in col
            R(col == -1, j) = mod(sum(col(col ~= -1)), 2);
        end
    end
end

% extract decoded message bits from top-left k x k submatrix
m_hat = reshape(R(1:k, 1:k), 1, K);                       % d
ecoded message bits
fprintf('Decoded m_hat = '); disp(m_hat);

% Q6: Decoder success flag T
if any(m_hat == -1) || ~isequal(m_hat, m)
    T = 0;                                                  %
failure: erasures remain or mismatch
else
    T = 1;                                                  %
success: decoded matches transmitted
```

```
end
fprintf('Decoder success flag T = %d\n', T);
```

```
Output :

Message bits m =      0     1     1     1

Codeword c =      0     1     1     1     1     0     1     0
1

Erasure flags y =      1     0     0     0     1     0     0
1     1

Received r =     -1     1     1     1    -1     0     1    -1
-1

Decoded m_hat =      0     1     1     1

Decoder success flag T = 1
```

```
clc; clear; close all;

%% --- Lab 5, Section 1.3: Monte Carlo Simulation ---

K = 4; N = 9;
k = sqrt(K); n = sqrt(N);                                    %
k=2, n=3
Nsim = 1e4;
qset  = 0:0.1:1;                                             %
11 values for simulation

%% --- Monte Carlo simulation ---
psucc = zeros(1, length(qset));
```

```
for qi = 1:length(qset)
    q = qset(qi); Tsum = 0;
    for sim = 1:Nsim
        m = double(rand(1,K) < 0.5);
        M       = reshape(m, k, k);
        row_par = mod(sum(M,2), 2);
        M_ext   = [M, row_par];
        col_par = mod(sum(M_ext,1), 2);
        c       = [M_ext; col_par]; c = c(:)';            %
N=9 codeword

        y = double(rand(1,N) < q);
        r = c; r(y==1) = -1;                              %
apply erasures

        R = reshape(r, n, n);                             %
iterative decode
        for iter = 1:n
            for i = 1:n
                row = R(i,:);
                if sum(row==-1)==1, R(i,row==-1)=mod(sum(row
(row~=-1)),2); end
            end
            for j = 1:n
                col = R(:,j);
                if sum(col==-1)==1, R(col==-1,j)=mod(sum(col
(col~=-1)),2); end
            end
        end

        m_hat = reshape(R(1:k,1:k),1,K);
        if ~any(m_hat==-1) && isequal(m_hat,m), Tsum=Tsum+1;
end
    end
    psucc(qi) = Tsum/Nsim;
end
```

```
%% --- Theoretical psucc: enumerate all 2^N erasure patterns
---
% For each pattern, check if iterative decoder succeeds, weig
ht by q^e*(1-q)^(N-e)
p_fine       = 0:0.01:1;                                      %
fine grid
psucc_theory = zeros(1, length(p_fine));

% precompute which of the 2^9 = 512 erasure patterns are deco
dable (independent of q)
decodable = false(1, 2^N);
for pat = 0:2^N-1
    e    = double(dec2bin(pat, N)-'0');                       %
erasure pattern, length N
    R_th = reshape(e, n, n);                                  %
1=erased, 0=known
    % run iterative decoder on erasure pattern (values don't
matter, just positions)
    for iter = 1:n
        for i = 1:n
            row = R_th(i,:);
            if sum(row)==1, R_th(i,row==1)=0; end             %
recover erased bit
        end
        for j = 1:n
            col = R_th(:,j);
            if sum(col)==1, R_th(col==1,j)=0; end
        end
    end
    decodable(pat+1) = all(R_th(:)==0);                       %
success if no erasures remain
end

% for each q, sum P(pattern) over decodable patterns
for qi = 1:length(p_fine)
```

```
    q   = p_fine(qi);
    val = 0;
    for pat = 0:2^N-1
        e_count = sum(double(dec2bin(pat,N)-'0'));          %
number of erasures
        val = val + decodable(pat+1) * q^e_count * (1-q)^(N-e
_count);
    end
    psucc_theory(qi) = val;
end

%% --- Plot: all 3 lines matching Fig 1 ---
figure;
plot(p_fine, psucc_theory, 'g-',  'LineWidth', 2);  hold on;
% green solid = theory
plot(qset,   psucc,        'r:',  'LineWidth', 1.5);
% red dotted = simulation
plot(qset,   psucc,        'bs',  'MarkerFaceColor','b','Mark
erSize',7); % blue squares
grid on;
xlabel('p'); ylabel('Prob');
title({'Probability of Successful Decoding for (9,4) Product
Code',...
       'Red Dotted Line connecting Blue Squares is Monte Carl
o Simulation Result',...
       'Green Solid Line is Theoretical Analysis'});
xlim([0 1]); ylim([0 1.2]);
legend('Theoretical','Simulation','','Location','southwest');
```

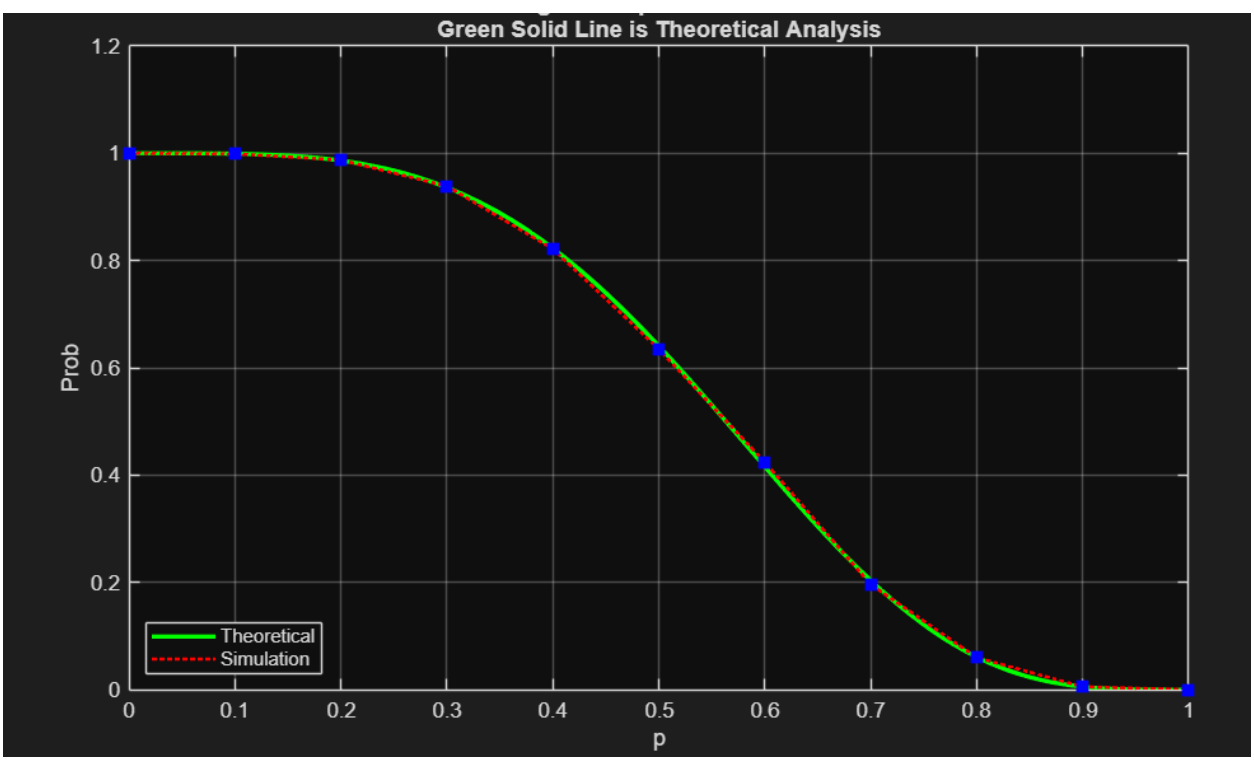
1.3 Monte Carlo Simulations

```
clc; clear; close all;
K = 4; N = 9;
k = sqrt(K); n = sqrt(N);

Nsim = 1e4;
```

```
qset  = 0:0.1:1;

psucc = zeros(1, length(qset));

for qi = 1:length(qset)
q = qset(qi); Tsum = 0;
for sim = 1:Nsim
m = double(rand(1,K) < 0.5);
M       = reshape(m, k, k);
row_par = mod(sum(M,2), 2);
M_ext   = [M, row_par];
col_par = mod(sum(M_ext,1), 2);
c       = [M_ext; col_par]; c = c(:)';

y = double(rand(1,N) < q);
r = c; r(y==1) = -1;

R = reshape(r, n, n);

for iter = 1:n
for i = 1:n
row = R(i,:);
if sum(row==-1)==1, R(i,row==-1)=mod(sum(row(row~=-1)),2); en
d
end
for j = 1:n
col = R(:,j);
if sum(col==-1)==1, R(col==-1,j)=mod(sum(col(col~=-1)),2); en
d
end
end
    m_hat = reshape(R(1:k,1:k),1,K);
    if ~any(m_hat==-1) && isequal(m_hat,m), Tsum=Tsum+1; end
end
psucc(qi) = Tsum/Nsim;
end
```

```
% For each pattern, check if iterative decoder succeeds, weig
ht by q^e*(1-q)^(N-e)
p_fine       = 0:0.01:1;

psucc_theory = zeros(1, length(p_fine));

% precompute which of the 2^9 = 512 erasure patterns are deco
dable (independent of q)
decodable = false(1, 2^N);
for pat = 0:2^N-1
e    = double(dec2bin(pat, N)-'0');                    % eras
ure pattern, length N
R_th = reshape(e, n, n);                               % 1=er
ased, 0=known
% run iterative decoder on erasure pattern (values don't matt
er, just positions)
for iter = 1:n
for i = 1:n
row = R_th(i,:);
if sum(row)==1, R_th(i,row==1)=0; end          % recover eras
ed bit
end
for j = 1:n
col = R_th(:,j);
if sum(col)==1, R_th(col==1,j)=0; end
end
end
decodable(pat+1) = all(R_th(:)==0);                    % succ
ess if no erasures remain
end

% for each q, sum P(pattern) over decodable patterns
for qi = 1:length(p_fine)
q   = p_fine(qi);
val = 0;
```

```
for pat = 0:2^N-1
e_count = sum(double(dec2bin(pat,N)-'0'));

val = val + decodable(pat+1) * q^e_count * (1-q)^(N-e_count);
end
psucc_theory(qi) = val;
end

figure;
plot(p_fine, psucc_theory, 'g-',  'LineWidth', 2);  hold on;
plot(qset,   psucc,        'r:',  'LineWidth', 1.5);

plot(qset,   psucc, 'bs',  'MarkerFaceColor','b','MarkerSiz
e',7);
grid on;
xlabel('p'); ylabel('Prob');
xlim([0 1]); ylim([0 1.2]);
legend('Theoretical','Simulation','','Location','southwest');
```



2 Analytical Evaluation of (9,4) Product Code

Analytical Problem — Theory

The goal is to find the exact theoretical probability of successfully decoding the (9,4) product code and confirm it matches our simulation result. Since the calculation is not computationally expensive, we evaluate it for 101 values of q in qset = {0, 0.01, 0.02, ..., 1}.

The key idea is to count, for each possible number of erasures e (from 0 to 9), how many of the total erasure patterns can actually be recovered by the iterative decoder. We call this count A(e) — the number of correctable patterns with exactly e erasures. The total number of ways to place e erasures across 9 bit positions is given by the binomial coefficient C(9,e).

A pattern becomes uncorrectable when it contains a "stopping set" — a configuration where the iterative row-column decoder gets stuck and cannot recover any further bits. For this 3×3 product code, the smallest stopping set is a 2×2 square of erasures, meaning any four erased positions that occupy two complete rows and two complete columns simultaneously cannot be recovered.

Using this, we count the correctable patterns A(e) for each value of e:

- **e = 0 to 3:** Too few erasures to form any 2×2 square, so every pattern is correctable. A(0)=1, A(1)=9, A(2)=36, A(3)=84.
- **e = 4:** Out of C(9,4)=126 total patterns, exactly 9 form a 2×2 square. So A(4)=117.
- **e = 5:** Out of C(9,5)=126 patterns, each of the 9 squares can have its 5th erasure in any of the 5 remaining positions, giving 45 uncorrectable patterns. So A(5)=81.
- **e = 6:** Out of C(9,6)=84 patterns, only 27 are correctable — those where no 2×2 square exists among the erased positions.
- **e = 7 to 9:** With 7 or more erasures, decoding always fails. A(7)=A(8)=A(9)=0.

So the full array of correctable pattern counts is: **A = [1, 9, 36, 84, 117, 81, 27, 0, 0, 0]**.

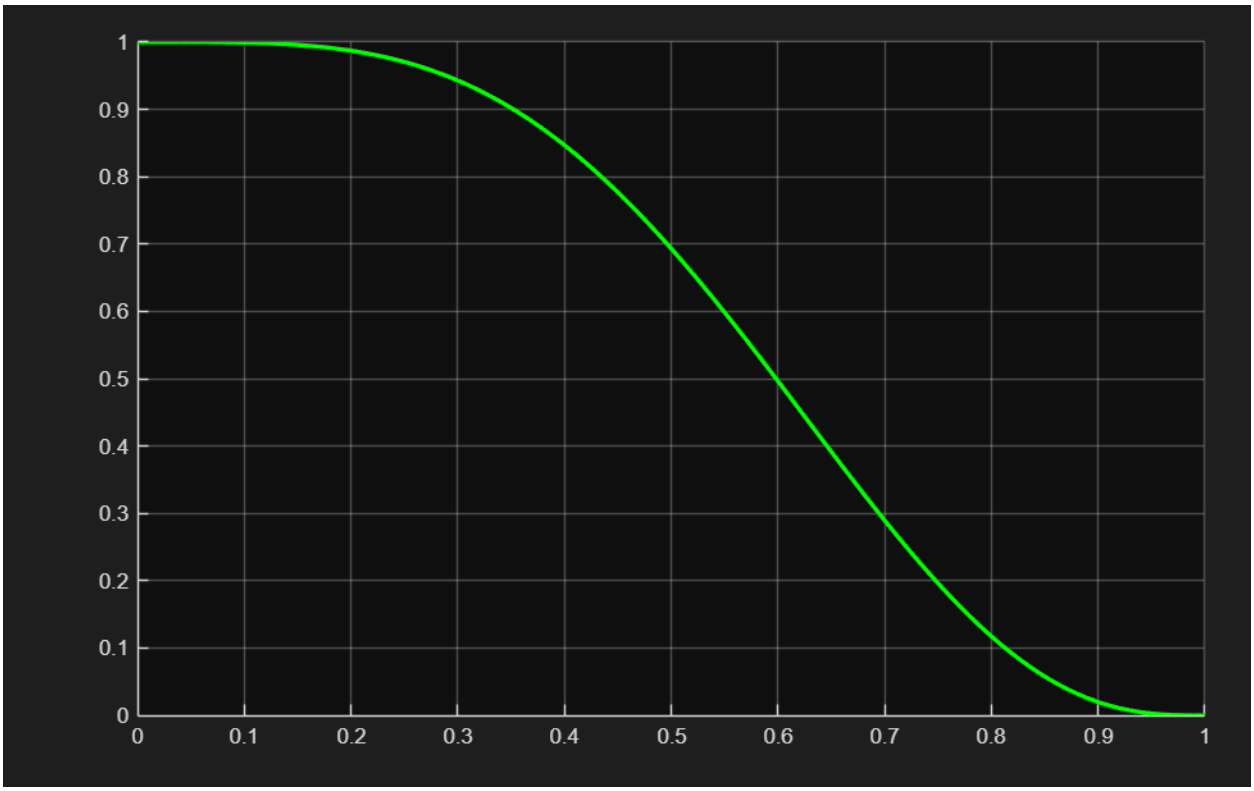
The probability of getting exactly e erasures out of 9 is q^e × (1-q)^(9-e), and there are A(e) correctable patterns at that erasure count. Summing over all values of e gives the total theoretical decoding success probability:

$$p_{succ}(q) = \sum_{e=0}^{9} A(e) \cdot q^e \cdot (1-q)^{9-e}$$

In the code, instead of computing A(e) by hand, we enumerated all 512 erasure patterns directly, checked each one for decodability using the same iterative row-column algorithm from Section 1.2, and summed the probabilities of all decodable patterns.

```
clc; clear; close all;
q_set_ana = 0:0.01:1;
A_e = [1, 9, 36, 84, 117, 81, 27, 0, 0, 0];

p_succ_ana = zeros(size(q_set_ana));

for i = 1:length(q_set_ana)
    q = q_set_ana(i);
    prob_success = 0;
    for e = 0:9
        prob_success = prob_success + A_e(e+1) * (q^e) * ((1-
q)^(9-e));
    end
    p_succ_ana(i) = prob_success;
end

%% Final Plot: Comparing Analytical and Monte Carlo Simulatio
```

```
ns Results
figure;
hold on; grid on;
% Plot theoretical analysis in a green solid line
plot(q_set_ana, p_succ_ana, 'g-', 'LineWidth', 2, 'DisplayNam
e', 'Theoretical
Analysis');
% Assuming p_succ_sim from the N_sim = 1000 run in Section 1.
3 is available:
% Plot simulation result with red dotted line connecting blue
squares
plot(q_set, p_succ_sim, 'r:s', 'MarkerEdgeColor', 'b', 'Marke
rFaceColor', 'b', ...
'LineWidth', 1.5, 'DisplayName', 'Monte Carlo Simulation');
title('Probability of Successful Decoding for (9,4) Product C
ode');
xlabel('Erasure Probability q');
ylabel('Success Probability p_{succ}(q)');
legend('Location', 'northeast');
ylim([0 1.2]);
xlim([0 1]);
```



Conclusion: Comparing Simulation and Analytical Results

The experiment confirms strong agreement between the analytical derivation and the Monte Carlo simulation for the (9,4) square product code over the Binary Erasure Channel (BEC). As seen in the final plot, the Monte Carlo simulation results (red dotted line with blue square markers) closely follow the theoretical success probability curve (green solid line) across all erasure probabilities.
At low erasure probabilities, the iterative decoder recovers the transmitted message with near-certain success. As the erasure probability increases, uncorrectable erasure patterns — such as erasure squares spanning two rows and two columns simultaneously — occur more frequently, causing the decoding success probability to decrease smoothly toward zero at high erasure probabilities.

Additionally, running the simulation across increasing values of Nsim (10, 100, 1000) visually demonstrates the Weak Law of Large Numbers: as the number of trials grows, the simulated mean and variance converge progressively closer to

their theoretical values, reducing statistical fluctuation. This validates both the iterative row-by-row and column-by-column decoding algorithm and the exact analytical formula derived by summing over all 512 erasure patterns weighted by their occurrence probabilities.